Report

OWASP MOBILE TOP 10

Suyash A Gunjal

What is OWASP mobile top 10?

OWASP Mobile Top 10 is a security awareness framework created to explain the most critical and common security risks found in mobile applications (Android and iOS).

It is not a testing tool, not a checklist, and not a compliance rule.
Instead, it is a knowledge guide that helps developers, testers, and security teams understand how mobile apps are commonly attacked and broken.

List of OWASP Top 10 Mobile Risks 2024

M1: Improper Credential Usage
M2: Inadequate Supply Chain Security
M3: Insecure Authentication/Authorization
M4: Insufficient Input/Output Validation
M5: Insecure Communication
M6: Inadequate Privacy Controls
M7: Insufficient Binary Protections
M8: Security Misconfiguration
M9: Insecure Data Storage
M10: Insufficient Cryptography

Comparison Between 2016-2024

OWASP-2016	OWASP-2024-Final Release	Comparison Between 2016-2024
M1: Improper Platform Usage	M1: Improper Credential Usage	New
M2: Insecure Data Storage	M2: Inadequate Supply Chain Security	New
M3: Insecure Communication	M3: Insecure Authentication/Authorization	Merged M4&M6 to M3
M4: Insecure Authentication	M4: Insufficient Input/Output Validation	New
M5: Insufficient Cryptography	M5: Insecure Communication	Moved from M3 to M5
M6: Insecure Authorization	M6: Inadequate Privacy Controls	New
M7: Client Code Quality	M7: Insufficient Binary Protections	Merged M8&M9 to M7
M8: Code Tampering	M8: Security Misconfiguration	Rewording [M10]
M9: Reverse Engineering	M9: Insecure Data Storage	Moved from M2 to M9
M10: Extraneous Functionality	M10: Insufficient Cryptography	Moved from M5 to M10

M1: Improper Credential Usage

This vulnerability relates to hardcoded credentials, insecure credential transmission and insecure credential storage. An example of this vulnerability is the British Airways breach.

How to detect it:

- Review application code and configuration files for Hardcoded Credentials.
- Review credential management for both third-party services and mobile applications.
- Inspect network traffic for secure credential transmission.

How to prevent it:

- Do not hardcode credentials into the app source code.
- Utilize tools like Spectral to scan your system for vulnerabilities.

M2: Inadequate Supply Chain Security

This risk targets SDKs, third-party libraries, vendors, credentials as well as any external systems the app interacts with. If the supply chain is compromised it could allow threat actors to inject malicious code, create backdoors and even take over the mobile app.

How to detect it:

- Vet third-party libraries/solutions.
- Include SAST and DAST testing in the development process.
- Use Software Composition Analysis tools to scan third-party libraries for vulnerabilities.

How to prevent it:

- Implement security controls for app updates, patches and releases.
- Establish a vetting process for third-party libraries.

M3: Insecure Authentication/Authorization

Once threat actors find any vulnerabilities in either the authentication or authorization scheme, they can exploit it in two ways:

- Bypass the authentication and submit requests to the backend server.
- Authenticate into the application but utilize weak endpoint authorization to gain access.

How to detect it:

- Look out for anonymous backend API execution.
- Test backend functionality (POST/GET) access with least-privileged roles to verify authorization weak points.

How to prevent it:

- Reinforce server-side authentication and authorization.
- Ensure that mobile applications ported from web applications follow the same authentication process, maintaining consistent web application security.

M4: Insufficient Input/Output Validation

Failing to validate and sanitize input/output data, puts your application at risk of being exploited by attacks: **SQL injections, Command Injections etc as well as your application inadvertently exposing confidential data**.

How to detect it:

- Perform Input/Output validation
- Test for data integrity.

How to prevent it:

- Use context-specific validation.
- Use tools like Sonarqube, Spectral, and Veracode to perform SAST/DAST.

M5: Insecure Communication

Most modern applications use some level of cryptographic protocols. However, there may be flaws in implementation: depreciated

protocols, accepting bad SSL certificates or inconsistencies (partially encrypted workflows). Any of these flaws could allow threat actors to intercept communications.

How to detect it:

- Identify application traffic going through plaintext channels.
- Check for bad SSL certificates (self-signed, revoked, expired, wrong host…)

How to prevent it:

- Implement up-to-date TLS/SSL on communication channels.
- Use certificates signed by a trusted CA provider.

M6: Inadequate Privacy Controls

What it is:

This vulnerability focuses on how this PII is stored and transmitted by your mobile application. Breaches resulting in the theft of PII not only affect your users but also have heavy regulatory and financial consequences.

How to detect it:

- Check the app for compliance violations.
- Review all systems in the app that use PII, for vulnerabilities.

How to prevent it:

- Perform routine privacy assessments on your application.
- Require only necessary data from users.

M7: Insufficient Binary Protections

What it is:

All mobile applications are vulnerable to binary attacks. Binary attacks can occur in two types: reverse engineering(decompiling and scanning the application for secrets, algorithms and vulnerabilities) and code tampering (removing licenses, circumventing paywalls…).

How to detect it:

- Use automated tools to perform SAST/DAST on your application.
- Verify the binary's digital signature integrity.

How to prevent it:

- Encrypt sensitive parts of your app binary code.
- Implement Runtime Application Self-Protection(RASP) solutions.
- Implement binary obfuscation.

M8: Security Misconfiguration

"**Security Misconfiguration**" is the eighth vulnerability in the "OWASP Mobile Top 10". This vulnerability exploits mistakes in configuration due to human error.

How to detect it:

- Review application source code.
- Identify security bottlenecks (default usernames, unencrypted storage and communication protocols…)
- Identify any testing features/systems in production.

How to prevent it:

- Implement the principle of least privilege, granting access to only what is needed.
- Utilize automated security tools to scan for security misconfigurations and vulnerabilities.

M9: Insecure Data Storage

What it is:

From poor data encryption to insecure secure data storage practices. This risk exploits vulnerabilities that could expose stored data.

How to detect it:

- Identify poor access controls in your system.
- Review stored data or unencrypted parts.

How to prevent it:

- Implement strong encryption.
- Implement secure session management techniques.
- Employ proper access controls.

M10: Insufficient Cryptography

What it is:

This risk exploits vulnerabilities that expose stored your mobile application, through flaws in cryptographic implementation.

How to detect it:

- Identify weak encryption algorithms in your application
- Review implementation of cryptographic protocols.
- Review key management for your application.

How to prevent it:

- Implement robust encryption algorithms.
- Pair security tools with scanning tools like Spectral to achieve all-round coverage.
- Follow secure key management practices.
- Avoid customized encryption.

References:

https://www.indusface.com/blog/owasp-mobile-top-10-2024/

https://brightsec.com/blog/owasp-mobile-top-10/